RLHF for Open-Ended Instruction Following: Offline Preference Optimization, Reward-Model RL, and What a Reward Model Does Not Measure

CS 185/285 Final Project (LLM RL), Spring 2026

> **Evaluation note (please read first).** The official metric of this project is the win rate against the frozen base model as judged by GPT-5.4 through the OpenAI API. I had no API access, so all win rates in this report come from a **local proxy judge** (`microsoft/phi-4`, 14B parameters, run on one GPU) that follows the same protocol. Section 3.3 validates this judge against the GPT-5.4 labels that ship with the benchmark. The proxy win rates are **not comparable with the handout's Gradescope thresholds**, and the proxy judge has a measurable bias towards longer responses, which matters for several conclusions below. The reward-model pair accuracy needs no judge and is exact.

Abstract

I implement three offline preference-optimization methods (DPO, IPO, AOT), a Bradley–Terry reward model, and three reward-model-based online methods (GRPO, DrGRPO, GSPO) for a Qwen2.5-1.5B-Instruct policy with LoRA adapters, and evaluate them head-to-head against the base model with a local proxy judge. The reward model reaches 0.83 pair accuracy on held-out pairs. Among the Part 1 methods, AOT is best under the proxy judge (0.80 ± 0.04) and DrGRPO is worst (0.38): DrGRPO obtains the *highest* reward-model score of all Part 1 policies while losing to the base model, because it shortens its responses. In Part 2 I test three changes to the DPO objective (confidence-aware weighting, a DPO-Positive penalty, label smoothing), reward-model reranking over the Part 1 offline policies, and two changes to GRPO (rank-only advantages and a pessimistic reward-model ensemble) with longer training. None of the offline ideas beats Part 1 AOT. Online, training three times longer raises GRPO from 0.58 to 0.68; rank-only advantages reach the same final win rate but remove the KL spikes (maximum sampled KL 0.08 instead of 3.6) and give a monotone improvement. The main lesson is that reward-model score and held-out preference accuracy were poor predictors of the judged win rate in my experiments.

1. Introduction

The benchmark asks for a policy that answers open-ended user prompts better than the base model, where "better" is decided by an LLM judge that compares two responses. The training signal is indirect: 4,744 preference pairs (prompt, chosen response, rejected response) labelled by GPT-5.4, and 4,744 unlabelled prompts for online rollouts. Offline methods learn directly from the pairs. Online methods first fit a reward model to the pairs and then optimize the policy against it with on-policy samples.

The questions of this report are: (1) how do the six required methods compare under one evaluation protocol; (2) does a reward model with good held-out accuracy give a reward that improves judged quality; and (3) can simple, motivated changes of the objective, of the reward-model usage or of the advantage computation improve the win rate.

2. Methods

Notation. x is a prompt, y^+ and y^- the chosen and rejected responses, π_θ the policy (base model plus a trainable LoRA adapter) and π_{ref} the frozen base model. $\log \pi(y \mid x)$ is the sum of token log-probabilities of the response.

2.1 Offline methods (Part 1)

All three use the reference-corrected margin

$$\Delta_\theta(x, y^+, y^-) = \big[\log \pi_\theta(y^+ \mid x) - \log \pi_\theta(y^- \mid x)\big] - \big[\log \pi_{\text{ref}}(y^+ \mid x) - \log \pi_{\text{ref}}(y^- \mid x)\big] .$$

- **DPO**: $L = -\log \sigma(\beta \Delta_\theta)$. A logistic loss on each pair; the margin is pushed up without bound, with a gradient that vanishes only when the pair is already classified with a large margin.
- **IPO**: $L = (\Delta_\theta - \frac{1}{2\beta})^2$. A regression of each pair's margin onto a fixed target; margins above the target are pulled back down, which bounds the deviation from the reference.

- **AOT**: with $r_\theta(y) = \log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)$, the chosen and rejected rewards of a minibatch are sorted separately and compared by rank: $L = -\frac{1}{B} \sum_i \log \sigma \left(\beta \left[r^+_{(i)} - r^-_{(i)} \right] \right)$. This asks the *distribution* of chosen rewards to dominate the distribution of rejected rewards (first-order stochastic dominance), comparing responses across different prompts.

2.2 Reward model

A scalar head on Qwen2.5-1.5B-Instruct (LoRA) scores a (prompt, response) pair, $s_\phi(x, y)$, and is trained with the Bradley–Terry loss $-\log \sigma \left(s_\phi(x, y^+) - s_\phi(x, y^-) \right)$.

2.3 Online methods (Part 1)

For each prompt, $G = 4$ responses are sampled, scored by the reward model, and centred by the group mean; GRPO and GSPO also divide by the group standard deviation, DrGRPO does not. All three maximise a clipped importance-weighted surrogate plus a sampled-token KL penalty to π_{ref}, and differ in how tokens are aggregated:

- **GRPO**: token-level ratios ρ_t, clipped per token, and averaged *within each sequence* before averaging over sequences. Every response has the same weight regardless of its length.
- **DrGRPO**: the same token-level surrogate, but summed over all tokens of the minibatch and divided by the total token count, with no per-sequence normalisation. A response contributes in proportion to its length.
- **GSPO**: one ratio per sequence, $\rho = \exp\left(\frac{1}{|y|} \sum_t \log \rho_t \right)$, clipped at the sequence level.

The clipped-surrogate loop (minibatching, KL penalty, gradient clipping) is shared by the three algorithms in `rl/clipped.py`; each algorithm only defines its surrogate.

2.4 Part 2 ideas

Offline. Three changes to DPO, selectable independently (`--algo dpo_plus`, `--example_weighting`, `--dpop_lambda`, `--label_smoothing`):

1. **Confidence-aware pair weighting.** The dataset stores the labelling judge's confidence $c \in [0.67, 0.99]$ and preference strength $s \in [2.5, 5]$ for each pair. The per-pair loss is weighted by $w = (2c - 1) \cdot s/5$ and normalised by the weight sum. A label with confidence c is correct with probability about c, so $2c - 1$ is its expected signed value; weak pairs keep a non-zero weight. *Motivation:* 15% of the pairs have strength ≤ 3, and DPO's unbounded margin keeps fitting them.
2. **DPO-Positive penalty** (Pal et al., 2024). Δ_θ is replaced by $\Delta_\theta - \lambda \max\left(0, \log \pi_{\text{ref}}(y^+ \mid x) - \log \pi_\theta(y^+ \mid x) \right)$ with $\lambda = 2$. *Motivation:* DPO can increase the margin by lowering the likelihood of *both* responses, as long as the rejected one falls faster; the penalty forbids lowering the chosen response.
3. **Label smoothing**, $L = -(1 - \epsilon) \log \sigma(\beta \Delta_\theta) - \epsilon \log \sigma(-\beta \Delta_\theta)$ with $\epsilon = 0.1$, which bounds the optimal margin for noisy labels.
4. **Reward-model reranking** (`rerank.py`). For every prompt, the greedy responses of the three Part 1 offline policies (DPO, IPO, AOT) are scored by the reward model and the best one is kept.

Online (options of `train_rm_grpo.py`):

5. **Rank-only advantages** (`--advantage_mode rank`). Within each group the four samples receive the advantages $(-1, -\frac{1}{3}, \frac{1}{3}, 1)$ in the order of their reward. *Motivation:* the reward model's scores are heavy-tailed (held-out margin standard deviation 5.6, larger than its mean 4.9); with mean/std normalisation a single outlier determines the advantages of its group, and a group with almost equal rewards is blown up to unit scale. Ranks use only the ordering, which is what a Bradley–Terry model is trained to get right.
6. **Pessimistic reward-model ensemble** (`--extra_reward_adapter_paths`, `--reward_pessimism`). The reward is the mean minus one standard deviation of the scores of three checkpoints of the reward-model run (steps 250, 350, 445; held-out accuracy 0.79, 0.82, 0.82). *Motivation:* responses on which the checkpoints disagree are where reward hacking is most likely.
7. **Longer training**: 75 instead of 25 iterations, with everything else unchanged, as the control for 5 and 6.

3. Experimental setup

3.1 Benchmark and model

Base model `Qwen/Qwen2.5-1.5B-Instruct`; LoRA rank 32 on all attention and MLP projections. Dataset `wildchat_min4_judged_5k_v1`: 4,744 training pairs (`train_prefs`), 256 held-out pairs (`test_prefs`), 4,744 rollout prompts (`train_gen`). Public evaluation files: 128 prompts with the base model's responses, and 256 labelled pairs. The benchmark settings are unchanged everywhere: `max_prompt_tokens = 700`, `max_new_tokens = 256`, `max_response_tokens = 512`, and greedy decoding for all evaluated outputs. No data beyond the repository is used.

3.2 Training

All hyperparameters are those of the handout commands; training ran locally on NVIDIA RTX A6000 GPUs (48 GB) by calling the training modules directly instead of the Modal entry points.

Run	Main settings	Evaluated checkpoint
Reward model	3 epochs (445 steps), batch 8 × 4 accumulation, lr 3e-5	step 445
DPO	$\beta = 0.1$, 3 epochs (890 steps), batch 4 × 4 accumulation, lr 5e-5	step 100
IPO	$\beta = 0.1$, otherwise as DPO	step 300
AOT	$\beta = 0.2$, otherwise as DPO	step 550
GRPO / DrGRPO / GSPO	25 iterations, 16 prompts × 4 samples, 2 PPO epochs, clip 0.2, KL coefficient 0.01, lr 1e-5, temperature 0.8	iteration 25
Part 2 offline	as DPO, 1 epoch (297 steps)	steps 100, 200, 297
Part 2 online	as GRPO, 75 iterations	iterations 25, 50, 75

The evaluated Part 1 checkpoints are the ones recommended in the handout.

3.3 Proxy judge and its validation

`llm_rl_final_proj/local_judge.py` uses the judge instructions and the pair prompt of the course autograder. Each pair is judged in both A/B orders. Instead of parsing generated text, the script reads the next-token probabilities of the labels "A" and "B" after the forced answer prefix `{"best_label": "`, which is deterministic. Two win rates are computed:

- **Debiased (main metric):** every prompt counts; the log-odds of the two orders are added, so an additive position bias cancels. With 128 prompts the standard error is about 0.04.
- **Pair-agree:** the autograder's rule; a prompt counts only if both orders pick the same response.

Validation on the 256 public pairs labelled by GPT-5.4 (chosen versus rejected response):

Quantity	Value
Agreement with GPT-5.4, debiased, all 256 pairs	0.820 ± 0.024
Agreement with GPT-5.4, pair-agree rule	0.892 on 148 of 256 pairs
Fraction of judgments that pick the first-listed response	0.70
Agreement when the GPT-5.4-preferred response is the *longer* one (133 pairs)	0.947
Agreement when the GPT-5.4-preferred response is the *shorter* one (123 pairs)	0.683
Base model against itself	0.50 by construction (no pair-agree rows)

Three properties of the proxy follow. (i) It agrees with GPT-5.4 on about 82% of labelled pairs, so it carries real signal. (ii) It has a strong position bias: on policy-versus-base comparisons only 27–45% of the prompts pass the pair-agree rule, which is why the debiased rate over all prompts is the main metric and the pair-agree rate is

reported next to it. (iii) **It prefers longer responses more than GPT-5.4 does**: it is much more reliable when the better response is the longer one. Win rates of policies that lengthen their responses are therefore probably over-estimated and those of policies that shorten them under-estimated.

4. Results

4.1 Part 1

Method	Proxy win rate (debiased)	Pair-agree win rate (rows)	Mean reward-model score	Mean tokens	Responses at the 256-token limit
Base model	0.50	–	−5.32	186	46%
DPO (step 100)	0.672 ± 0.042	0.875 (40)	−2.23	192	41%
IPO (step 300)	0.645 ± 0.042	0.732 (41)	−4.21	203	55%
AOT (step 550)	**0.797 ± 0.036**	0.950 (40)	−4.62	241	78%
GRPO	0.582 ± 0.044	0.643 (42)	−3.26	183	42%
DrGRPO	0.383 ± 0.043	0.482 (56)	−1.88	144	17%
GSPO	0.574 ± 0.044	0.638 (47)	−2.09	167	32%

The reward model has a pair accuracy of **0.832** on the 256 public pairs (handout threshold 0.74). All policy numbers are on the 128 public prompts with greedy decoding. The reward-model score is the mean score of the same 128 responses under the step-445 reward model.

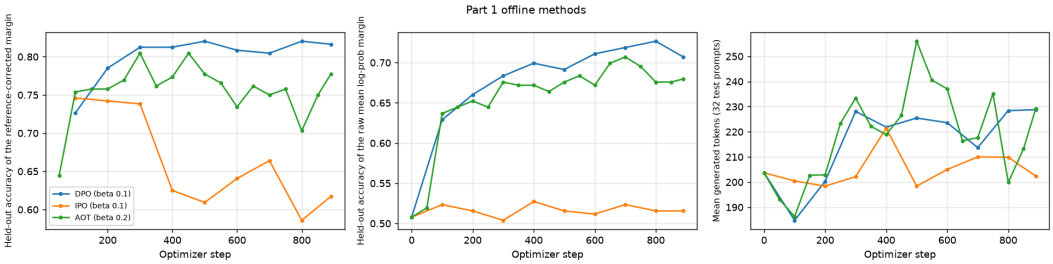


Figure 1: Offline methods: held-out preference accuracy and generation length during training.

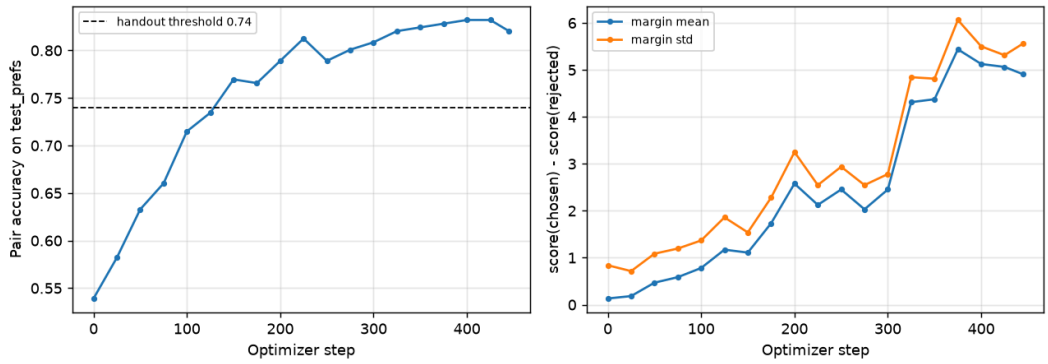


Figure 2: Reward model: held-out pair accuracy and margin.

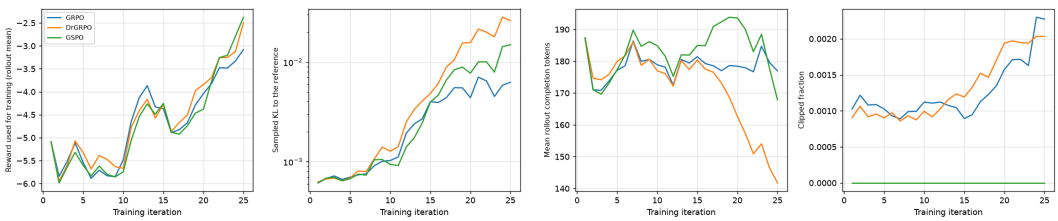


Figure 3: Online methods (Part 1): rollout reward, sampled KL, completion length and clip fraction.

Checkpoints. The win rate depends on the checkpoint as much as on the method:

Method	Step 100	Step 200	Step 300	Step 500–600	Step 890
DPO	0.672	0.711	0.781	0.734 (500)	0.754
IPO	0.684	–	0.645	0.562 (600)	0.641
AOT	0.648	–	0.812	0.797 (550)	0.719

DPO at step 300 is within noise of AOT. The ranking "AOT > DPO" in the main table is therefore partly a consequence of the recommended DPO checkpoint being early.

4.2 Answers to the Part 1 questions

1. Why do DPO, IPO and AOT optimize different notions of improvement? They share the data and the reference-corrected log-ratio, but impose different constraints on it. DPO is a per-pair classification: it is satisfied only when *each* pair has a large positive margin, and it never stops increasing margins. In my run the mean held-out margin grows from 23.8 nats at step 300 to 63.5 at step 890 while the accuracy stays at 0.81–0.82, so the later epochs only make the policy more extreme. IPO is a per-pair regression to the fixed target $1/(2\beta) = 5$ nats: improvement means "move every pair to the same moderate margin", which regularises towards the reference; its held-out margin stays at 1–2 nats and its raw preference accuracy stays at 0.50–0.53, so the policy changes little. AOT is a distributional criterion: the sorted chosen rewards should dominate the sorted rejected rewards. Since the pairing is by rank and not by prompt, AOT does not require every individual pair to be ordered correctly, which makes it less sensitive to single noisy pairs, but it also compares responses to different prompts.

2. Which offline method worked best, and what do the samples show? AOT (0.80), followed by DPO (0.67 at the recommended step, 0.78 at step 300) and IPO (0.65). AOT's responses are systematically longer and more "assistant-like": 241 tokens on average against 186 for the base model, 78% reach the token limit, and they often add an opening sentence and a closing explanation. For the prompt *"Write the 4-line poem about the my girfriend!"* the base model writes eight lines; AOT writes four lines and then a paragraph that explains the poem. The judge rewards this completeness. It is not always an improvement: for *"Write a Vince McMahon vs kevin Owen's with Owen's win in 20 seconds"*, AOT writes a longer, well-organised answer in which Vince McMahon wins, which contradicts the prompt. Given the length bias of the proxy judge (Section 3.3), AOT's margin over the other methods is probably smaller under GPT-5.4.

3. How did the reward model behave on held-out pairs, and did this line up with downstream quality? Its held-out pair accuracy rises quickly to 0.79 at step 200 and saturates at 0.82–0.83 from step 325, above the 0.74 threshold. The margin keeps growing after the accuracy has saturated (mean 2.5 at step 300, 4.9 at the end, with a standard deviation that is larger than the mean), i.e. the scores become more extreme without ranking more pairs correctly. Pair accuracy did **not** line up with downstream quality. Across all evaluated policies, the mean reward-model score is unrelated or even inversely related to the judged win rate:

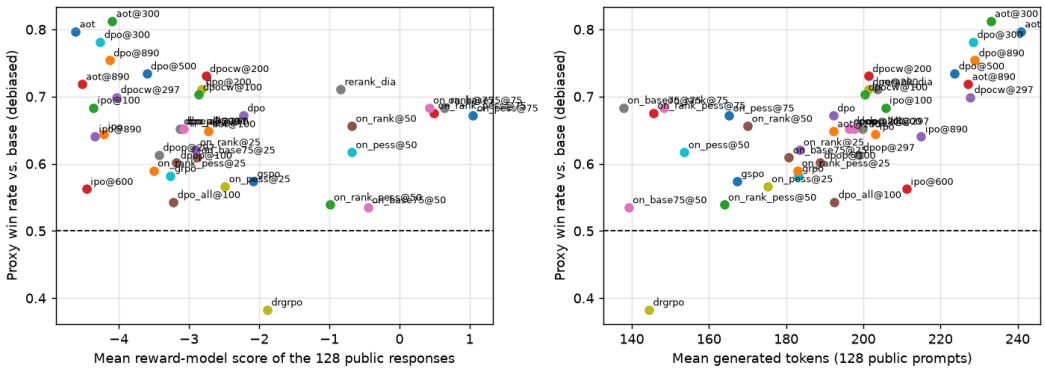


Figure 4: Proxy win rate against mean reward-model score (left) and against response length (right) for all evaluated checkpoints.

The policy with the highest reward-model score in Part 1 (DrGRPO, −1.88) is the only one that loses to the base model, and the policy with the lowest score (AOT, −4.62) wins most often. The reward model was trained

to separate chosen from rejected responses of *other* models for a prompt; it was never trained on Qwen-1.5B outputs. On this new distribution it prefers shorter, plainer responses (the right panel shows that the judge prefers the opposite), so maximising it moves the policy in a direction that the judge does not reward.

4. GRPO, DrGRPO, GSPO. GRPO (0.58) and GSPO (0.57) are equal within noise; DrGRPO (0.38) is clearly worse. All three increase the reward-model score, DrGRPO and GSPO the most (evaluation score −5.0 ⟶ −2.1 and −1.7; GRPO −3.2). The difference is what they change to get there. DrGRPO weights each response by its number of tokens and does not divide the advantages by the group standard deviation. Long responses dominate the gradient, and the unnormalised advantages are large exactly in groups with outlier scores. The cheapest way to reduce a large negative token-summed term is to make such responses shorter: the mean rollout length falls from 180 to 142 tokens during training (GRPO: 177 ⟶ 177) and the sampled KL is four times that of GRPO (0.023 versus 0.006). GRPO's per-sequence mean removes the length weighting and its updates stay small. GSPO lies in between (KL 0.011, length 168); its sequence-level ratio never left the clip range (clip fraction 0), so it behaved like an unclipped policy gradient with normalised advantages. Because the proxy judge favours longer responses, DrGRPO's deficit is probably overstated, but its direction (it is the only method below the base model) is consistent with the reward-model mismatch described above.

5. An internal metric that looked encouraging while the evaluation showed a problem. DrGRPO: within 25 iterations the mean reward-model score of its evaluation samples improved by 3.0 (the largest gain of the three online methods together with GSPO), no collapse diagnostic fired (no empty or repetitive generations), and the KL stayed at 0.02. Head-to-head it wins only 38% of the comparisons. The length statistic was the visible warning: the mean generation length fell by a third. A second example is IPO: its reference-corrected held-out accuracy is 0.74–0.75 at steps 100–300, similar to DPO's 0.73 at step 100, but its raw accuracy never leaves 0.50–0.53, and its win rate does not improve with training.

4.3 Part 2: offline

Method (1 epoch)	Step 100	Step 200	Step 297/300	Held-out corrected accuracy at the end	Held-out margin at the end (nats)
DPO (Part 1 run)	0.672	0.711	0.781	0.812	23.8
1. confidence weighting	0.703	**0.730**	0.699	0.812	24.4
1. DPO-Positive penalty (λ = 2)	0.602	0.652	0.613	0.738	7.4
1. weighting + penalty + label smoothing	0.543	0.652	0.652	0.742	5.2
Reward-model reranking over {DPO, IPO, AOT}	0.711 (single output; picks: DPO 58, IPO 34, AOT 36 prompts)				
Reference: Part 1 AOT (step 550)	*0.797*				

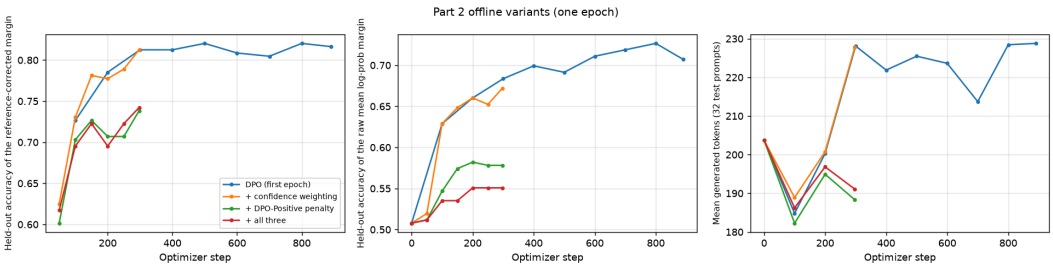


Figure 5: Part 2 offline variants.

- **Confidence weighting** changes little. It is slightly ahead of DPO at steps 100 and 200 (+0.03 and +0.02) and behind at the end of the epoch (−0.08); all three differences are below two standard errors (about 0.06 for a difference). Held-out accuracy and margin are the same as for DPO. The weights only range over a factor of about four and most pairs have similar weights, so the objective is close to DPO.
- **The DPO-Positive penalty and label smoothing hurt.** Both bound how far the policy moves: the final held-out margin is 5–7 nats instead of 24, the held-out accuracy 0.74 instead of 0.81, and the win rate 0.61–0.65 instead of 0.70–0.78. The hypothesis behind the penalty, that DPO gains its margin by harmfully lowering the likelihood of chosen responses, is not supported: the constrained policy stays closer to the base model and wins less.
- **Reward-model reranking fails.** The reranked output has a far higher reward-model score (−0.84) than any member of its candidate family (−2.2 to −4.6), and a win rate of 0.71, below the best member (AOT, 0.80) and close to the average of the three. The selector is the same misaligned reward model as in Section 4.2: it prefers the DPO response for 58 prompts and the AOT response for only 36.

`part2/offline_best.jsonl` is the confidence-weighted DPO policy at step 200 (0.730), the best Part 2 offline result. It does **not** beat the Part 1 AOT baseline (0.797) or DPO at step 300 (0.781).

4.4 Part 2: online

Method (GRPO base)	Iteration 25	Iteration 50	Iteration 75	Pair-agree at 75 (rows)	Maximum sampled KL in training	Mean tokens at 75
GRPO, 25 iterations (Part 1)	0.582	–	–	0.643 (42)	0.007	183
GRPO, 75 iterations	0.609	0.535	0.684	0.702 (47)	3.6 (iteration 47)	138
1. rank-only advantages	0.621	0.656	**0.684**	0.843 (51)	0.08	148
1. pessimistic ensemble	0.566	0.617	0.672	0.827 (52)	96.6 (one iteration)	165
1. rank + pessimistic ensemble	0.590	0.539	0.676	0.755 (49)	0.10	146

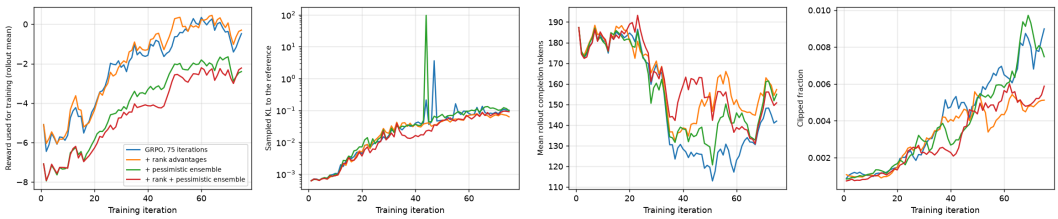


Figure 6: Part 2 online variants (KL on a logarithmic axis).

- **Longer training is what raises the win rate**: all four 75-iteration runs end at 0.67–0.68, against 0.58 for the 25-iteration Part 1 run (a difference of about 1.7 standard errors). The four variants are indistinguishable at the end.
- **Rank-only advantages make the optimisation stable.** With mean/std-normalised advantages, both runs without ranks have an iteration in which the sampled KL explodes (3.6 for plain GRPO, 96.6 for the pessimistic ensemble), and plain GRPO's win rate drops to 0.535 at iteration 50 before recovering. With ranks the KL never exceeds 0.10, the win rate rises monotonically (0.62, 0.66, 0.68), and the final pair-agree win rate is the highest (0.84 on 51 prompts, against 0.70 for plain GRPO). The evidence for a *higher* final win rate is weak; the evidence for a better-behaved training process is clear.
- **The pessimistic ensemble does not help.** It lowers the reward scale (the rollout reward is about 2 lower throughout) but not the direction of the update, and it does not prevent the KL spike. The three checkpoints come from one training run and are strongly correlated, so their disagreement is a weak uncertainty signal.

- All online runs still shorten the responses (138–165 tokens at iteration 75), i.e. they follow the reward model's preference for brevity; the reward-model score becomes positive (0.4 to 1.1) while the win rate saturates below 0.7.

`part2/online_best.jsonl` is GRPO with rank-only advantages at iteration 75 (0.684 ± 0.041).

4.5 Answers to the Part 2 questions

1. What did I try and why? Section 2.4 lists the seven ideas with their motivations: three robustness changes to DPO aimed at noisy or weak pairs and at DPO's unbounded margin, reranking with the reward model as a cheap way to combine three different offline policies, rank-only advantages and a pessimistic ensemble to make the online methods less sensitive to the heavy-tailed and partly unreliable reward-model scores, and longer online training as the control.

2. What improved the win rate, and where did the improvement come from? Only the online results improved over their Part 1 baseline: GRPO from 0.58 to 0.68. This gain comes from an *optimisation detail* (three times more iterations), not from the objective or from the reward-model usage: plain GRPO reaches the same value at iteration 75. Rank-only advantages changed the reliability of the path (no KL spike, monotone improvement), not the end point. Offline, the largest effect of all was *checkpoint choice* (DPO: 0.67 at step 100, 0.78 at step 300), which the handout rightly does not count as a method.

3. What failed, and what is the evidence? (a) The DPO-Positive penalty and label smoothing: lower win rate at every checkpoint (by 0.06–0.17), together with lower held-out accuracy and a three- to five-fold smaller margin. (b) Reward-model reranking: 0.71 against 0.80 for the best candidate, while the reward-model score of the selection is much higher than that of every candidate, which shows that the selector optimises something else than the judge. (c) The pessimistic ensemble: same final win rate as the control, and it did not prevent the KL spike. (d) Confidence weighting: no difference beyond noise.

4. Why did the strongest Part 2 method beat its Part 1 baselines? The strongest Part 2 result relative to its baseline is online (rank-advantage GRPO at iteration 75: 0.68, against 0.58, 0.57 and 0.38 for GRPO, GSPO and DrGRPO). It beats them mainly because it trains longer under a constraint that keeps every update small: with 25 iterations and a learning rate of 1e-5 the Part 1 policies have barely left the base model (KL below 0.01), and the judged quality keeps improving for at least 75 iterations. Ranks bound each advantage to [−1, 1] and remove the dependence on the reward model's score scale, which is what allows the longer run without the instability seen in the control. In absolute terms, however, the best policy of this project is still the Part 1 offline AOT policy (0.80); no Part 2 method exceeds it, and the online methods are limited by the reward model's misalignment.

5. With twice the budget. (i) Evaluate with the official GPT-5.4 judge, or at least with a second proxy judge and a length-controlled comparison, because the length bias of the proxy confounds the main comparison (AOT against the rest). (ii) Fix the reward model rather than the policy optimiser: add the policy's own samples to the reward-model training distribution (for instance, pairs of base-model responses ranked by the offline AOT policy's implicit reward), since every online result is bounded by the reward model's preference for short answers. (iii) Initialise online training from the AOT adapter and use the rank-advantage GRPO for fine-tuning with a KL penalty to the AOT policy. (iv) Run each configuration with more than one seed: with 128 prompts the standard error of a win rate is 0.04, and most differences in this report are within two standard errors.

5. Discussion

What worked. All Part 1 components behave as expected in their own metrics: the three offline losses raise held-out preference accuracy, the reward model reaches 0.83 pair accuracy, and the three online methods raise the reward-model score with bounded KL. Under the proxy judge, five of the six Part 1 policies beat the base model, and AOT is the strongest.

What did not. The internal metrics did not predict judged quality. The reward model's score is, across policies, anti-correlated with the win rate, and the offline held-out accuracy does not separate DPO at step 100 from DPO at step 300, although their win rates differ by 0.11. My Part 2 objective changes addressed problems (label

noise, falling chosen likelihood, reward-model outliers) that turned out not to be the limiting factor; the limiting factor is the mismatch between what the reward model scores highly and what the judge prefers.

Limitations. (i) All win rates are from a proxy judge with 82% agreement with GPT-5.4 on labelled pairs, a strong position bias and a bias towards longer responses; conclusions that depend on response length (AOT's lead, DrGRPO's deficit) are uncertain. (ii) One seed per configuration and 128 evaluation prompts. (iii) The Part 2 investigation produced no method that beats the best Part 1 baseline.

Appendix A: qualitative examples (greedy decoding)

Prompt: *Write the 4-line poem about the my girfriend!*

- **Base** (8 lines): "My dear friend, you light up my world, / With your laughter and love, I feel so free. / Your smile is like a ray of sunshine, / Guiding me through life's darkest hours. / Together we explore new horizons, / ..."
- **AOT**: "Here's a 4-line poem about your girlfriend: / My friend with eyes like stars, / Whose laughter fills the room so bright. / Her smile lights up every day, / In her presence, I feel at home. / This poem aims to capture the essence of your girlfriend - her sparkling eyes, infectious laugh, ... Feel free to adjust it ..."
- **DrGRPO** (4 lines): "My dear friend, you light up my world, / With your smile and laughter, I feel so free. / Your love and kindness, I will always treasure, / Together we'll face all life's challenges with grace."
- **GRPO with rank advantages, iteration 75** (4 lines): "My dear friend is like a shining star, / Guiding me through life's endless path. / Her smile lights up my world with joy, / And her love fills my heart with grace."

Prompt: *Give me a creative Ai art prompt for a snowflake ring not too long*

- **Base** (142 characters): "Create an ethereal snowflake ring with delicate frosty details and sparkling diamonds in the center, symbolizing purity and winter's beauty."
- **AOT** (505 characters): an introduction, a three-sentence prompt, and a closing sentence that says the prompt is short.
- **DrGRPO** (139 characters): "Create an ethereal snowflake ring with delicate icicles hanging from the center, surrounded by sparkling diamonds in a minimalist design."

The online policies follow explicit length constraints better than the base model; AOT adds framing text that the judge tends to reward even when the prompt asks for brevity.

Appendix B: commands

```
export WANDB_MODE=offline
D=dataset/wildchat_min4_judged_5k_v1; M=Qwen/Qwen2.5-1.5B-Instruct; R=runs/
wildchat_min4_judged_5k

# Reward model
uv run python -m llm_rl_final_proj.reward_model.train --model_name $M --dataset_name $D
--output_dir ${R}_reward_model_v1 --num_train_epochs 3 ...

# Offline (handout flags; --algo ipo and --algo aot --beta 0.2 for the other two)
uv run python -m llm_rl_final_proj.train --algo dpo --beta 0.1 --model_name $M --
dataset_name $D --output_dir ${R}_dpo_beta01_v1 \
  --per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 --num_train_epochs 3 --
max_prompt_tokens 700 --max_response_tokens 512 ...

# Online (handout flags; --algo dr_grpo / gspo for the other two)
uv run python -m llm_rl_final_proj.online.train_rm_grpo --algo grpo --model_name $M --
dataset_name $D \
  --reward_adapter_path ${R}_reward_model_v1/checkpoints/step_000445/adapter --
output_dir ${R}_grpo_rm445_v1 --steps 25 ...

# Part 2 offline
uv run python -m llm_rl_final_proj.train --algo dpo --example_weighting conf_strength --
num_train_epochs 1 ...
```

```
uv run python -m llm_rl_final_proj.train --algo dpo_plus --dpop_lambda 2.0 [--
label_smoothing 0.1 --example_weighting conf_strength] ...
uv run python -m llm_rl_final_proj.rerank --adapter_path <reward adapter> --candidates
<jsonl files> --select_from dpo ipo aot --output_jsonl ... --scores_json ...

# Part 2 online
uv run python -m llm_rl_final_proj.online.train_rm_grpo --algo grpo --steps 75 --
advantage_mode rank ...
uv run python -m llm_rl_final_proj.online.train_rm_grpo --algo grpo --steps 75 --
reward_pessimism 1.0 \
  --extra_reward_adapter_paths <step_000250 adapter>,<step_000350 adapter> ...

# Submission files and evaluation
uv run python -m llm_rl_final_proj.build_policy_submission --adapter_path <adapter> --
prompts_jsonl public_eval/public_test_gen_prompts_128.jsonl \
  --output_jsonl <out.jsonl> --max_prompt_tokens 700 --max_new_tokens 256 --temperature
0.0 --top_p 1.0
uv run python -m llm_rl_final_proj.local_judge --mode prefs  --output_dir runs/judge/
prefs_phi4
uv run python -m llm_rl_final_proj.local_judge --mode policy --output_dir runs/judge/
<name> --student_jsonl <out.jsonl> ...
uv run --with matplotlib python scripts/make_report.py
```

The complete commands with every flag are in the README of the code archive.